

# Optimization of Machine Learning Models

# Contents

Gradient Descent Algorithm(GD)

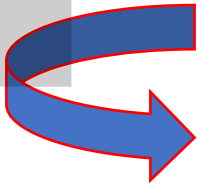
# Optimization Algorithms in Neural Network

- The process of minimizing (or maximizing) any mathematical expression is called **optimization**.
- Optimizers are algorithms or methods used to change the attributes of the neural network such as **weights** and **biases** to reduce the losses and to provide the most accurate results possible.

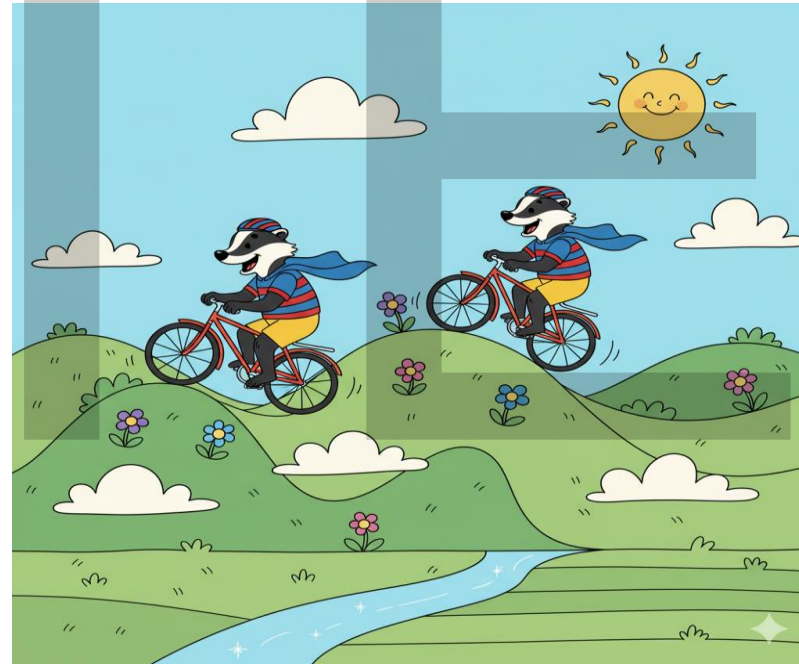
# Gradient Descent Algorithm

- **Optimization** algorithms in machine learning are mathematical techniques used to adjust a model's parameters to minimize errors and improve accuracy.
- Gradient Descent algorithm is heavily applied in Deep Learning

You want to  
descent from the  
top of a hill



How to choose the correct  
path?



# Gradient Descent Algorithm

- Gradient Descent (GD) is a first-order iterative minimization method.
- Iteratively updates parameters to minimize the Loss function.

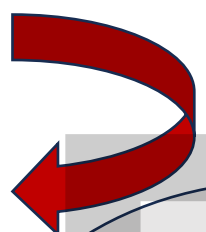
Uses **gradients (derivatives)** to decide the direction of change

Computes **partial derivatives** of the **loss function** with respect to every weight in a neural network, using the chain rule of calculus.

Derivative ?

Partial Derivative ?

Gradient?



**Loss function**  
returns the error  
between predicted  
outcomes and actual  
outcomes.

# Derivative, Partial Derivative and Gradient

Loss function:  $L(w) = 3w^2$

*Derivative:*  $\frac{dL}{dw} = 6w$

Loss function:  $L(w, b) = 3w^2 + b^2$

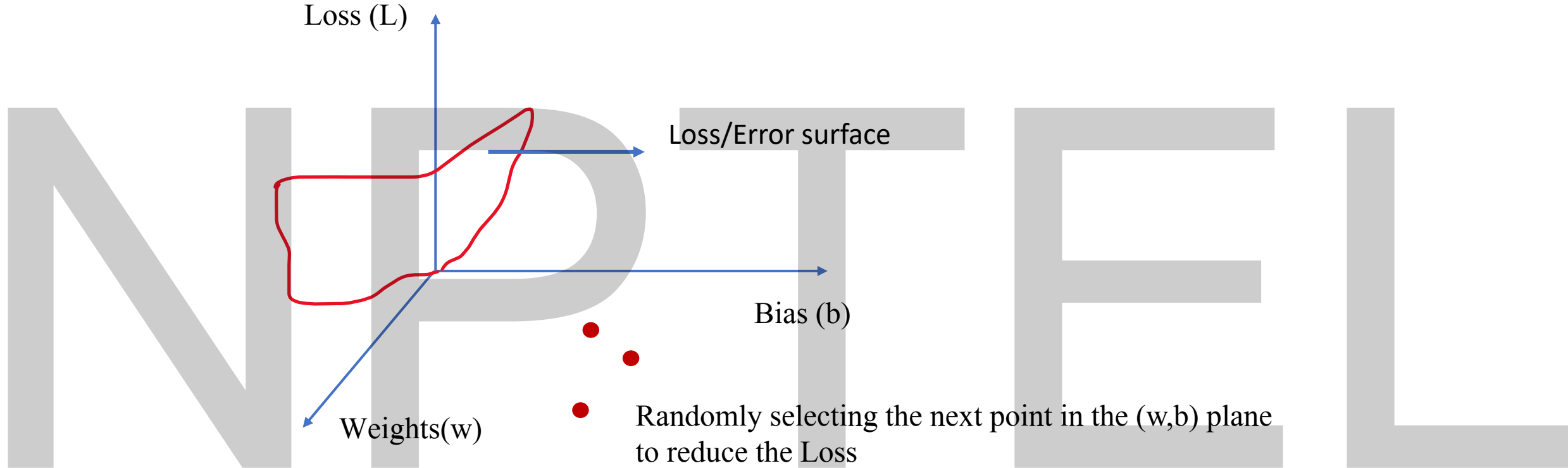
*Partial Derivative:*  $\frac{\partial L(w, b)}{\partial w} = 6w$

$$\frac{\partial L(w, b)}{\partial b} = 2b$$

*Gradient:*  $\begin{bmatrix} \frac{\partial L(w, b)}{\partial w} \\ \frac{\partial L(w, b)}{\partial b} \end{bmatrix} = \begin{bmatrix} 6w \\ 2b \end{bmatrix}$

Suppose I have a function with 10 variables, what will be the dimension of the gradient vector?

# Intuition of Gradient Descent Algorithm

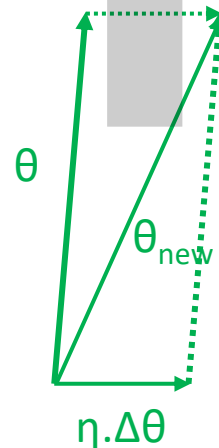
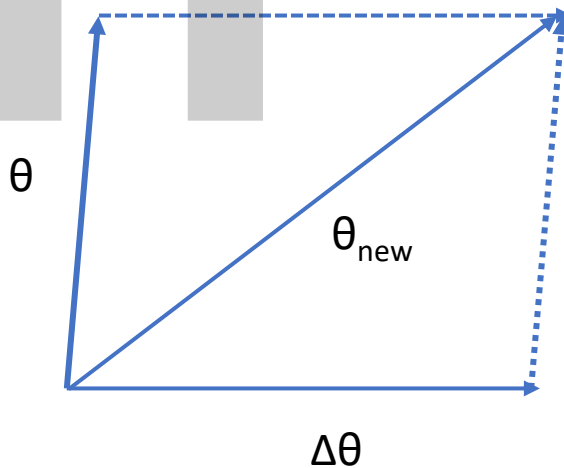


With Gradient Descent, we will get a structured way to move in a certain direction which will definitely reduce the Loss

# Gradient Descent Algorithm

- Let  $\theta = [w, b]$ , a vector of parameters of the neural network
- We start from a random value of weights and biases and calculate the loss function.
- We need to update these weights and biases so that the new loss function is lesser than the old loss function
- Let the change in  $\theta$  be  $\Delta\theta$

where  $\Delta\theta = [\Delta w, \Delta b]$



$$\theta_{\text{new}} = \theta + \eta \Delta\theta$$

Where  $\theta_{\text{new}} = [w_{\text{new}}, b_{\text{new}}]$

$\eta$  is a small value



# Which direction $\Delta\theta$ should be?

Old Loss =  $L(\theta)$

New Loss =  $L(\theta + \eta\Delta\theta)$

Let us call  $\Delta\theta = u$ ,  $L(\theta + \eta\Delta\theta) = L(\theta + \eta u)$

According to Taylor's series:

$$\begin{aligned} L(\theta + \eta u) &= L(\theta) + \eta * u^T \nabla L(\theta) + \frac{\eta^2}{2!} * u^T \nabla^2 L(\theta) . u + \frac{\eta^3}{3!} + \dots \\ &= L(\theta) + \underbrace{\eta * u^T \nabla L(\theta)}_{\text{negative}} \end{aligned}$$

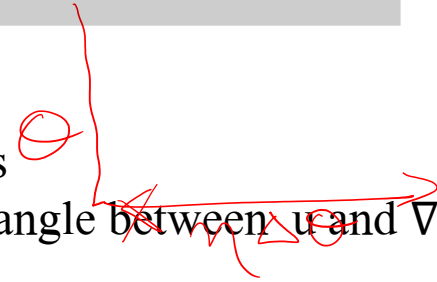
$L(\theta + \eta u) < L(\theta)$

$\eta * u^T \nabla L(\theta)$

$\eta$  cannot be negative

$u^T \nabla L(\theta)$  is the dot product between two vectors

This can become maximum negative when the angle between  $u$  and  $\nabla L(\theta)$  is  $180^\circ$



# Gradient Descent Algorithm

According to Gradient descent, if we want to reduce the loss, we should move in a direction which is opposite to the gradient of the loss

$$\underline{w^{(t+1)}} = \underline{w^{(t)}} - \eta \nabla_w L(w, b)$$

$$b^{(t+1)} = b^{(t)} - \eta \nabla_b L(w, b)$$

$$\begin{bmatrix} w^{(t+1)} \\ b^{(t+1)} \end{bmatrix} = \begin{bmatrix} w^{(t)} \\ b^{(t)} \end{bmatrix} - \eta \begin{bmatrix} \nabla_w L(w, b) \\ \nabla_b L(w, b) \end{bmatrix}$$

We are moving in a direction opposite to the gradient of the loss function

At  $w = w_t$  and  $b = b_t$

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta)$$

# Gradient Descent Algorithm

```
t ← 0;  
max_iter ← 500  
while t < max_iter do  
    w(t+1) ← wt - η∇wL(w, b)  
    b(t+1) ← bt - η∇bL(w, b)  
end
```

Max\_iter is taken to be a large value so that the iteration continue till it converges

# Learning rate in Gradient Descent

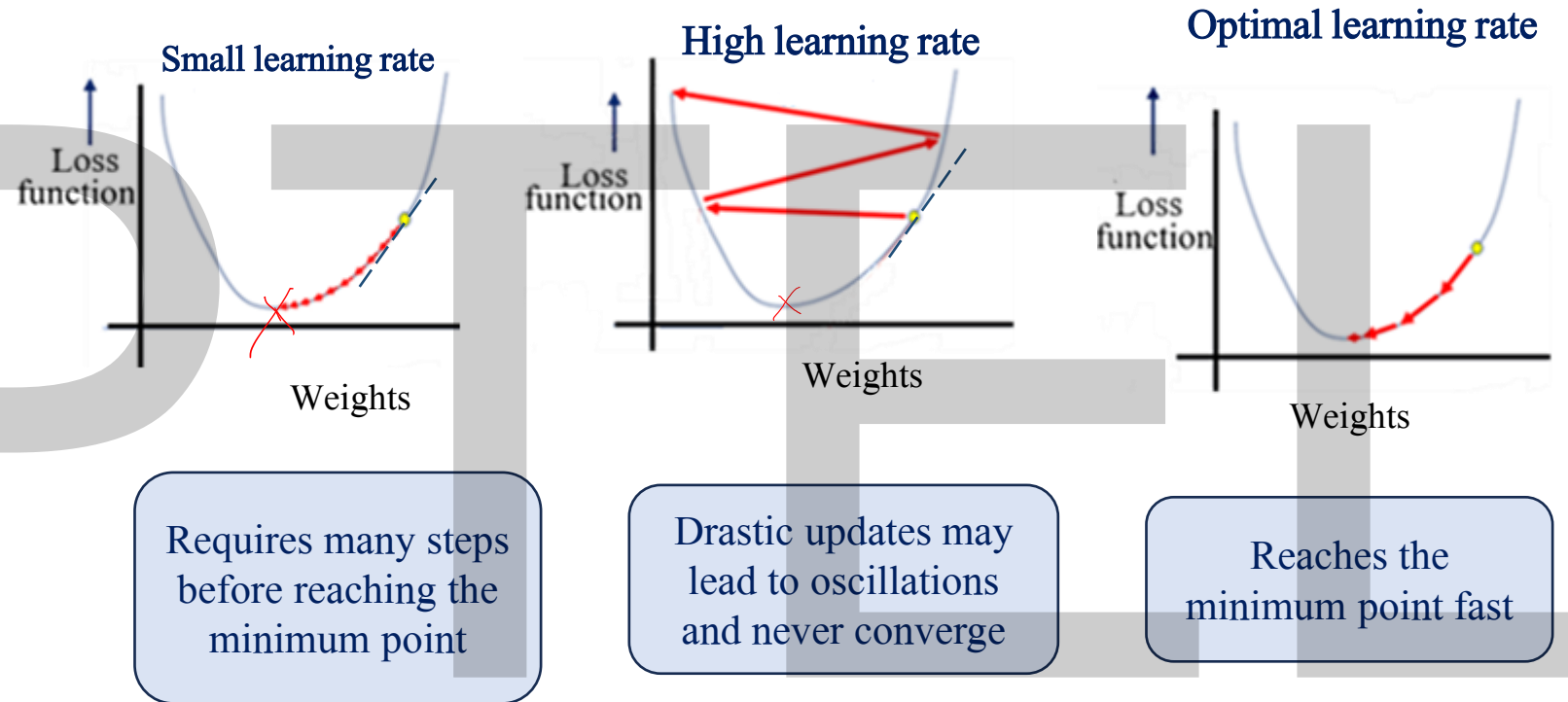
$$w^{(t+1)} = w^{(t)} - \eta \nabla_w L(w, b)$$

$\eta$  = Learning rate



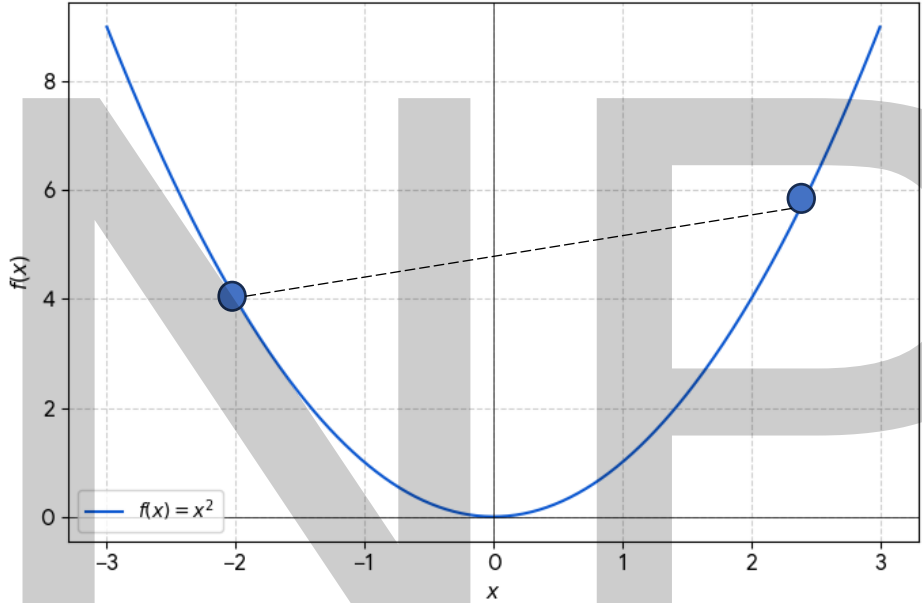
hyperparameter in NN that controls how quickly the model learns during training.

Usually  $\eta$  is chosen to be 0.01, 0.03..0.1, 0.3 ....etc



# Convex and Non Convex Loss function

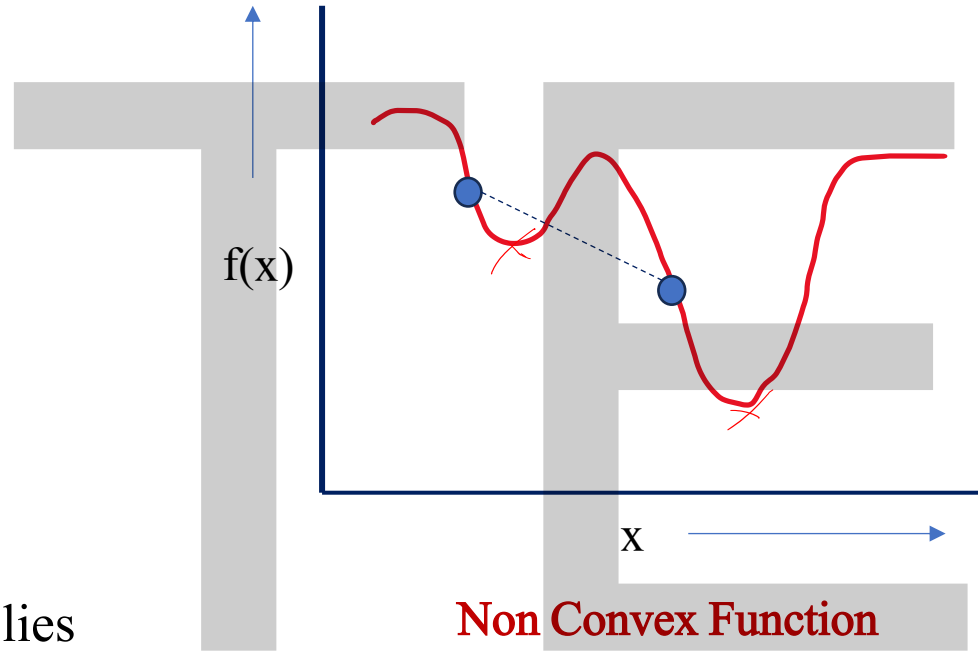
Graph of a Convex Function:  $f(x) = x^2$



A line connecting two points on the curve always lies above the curve

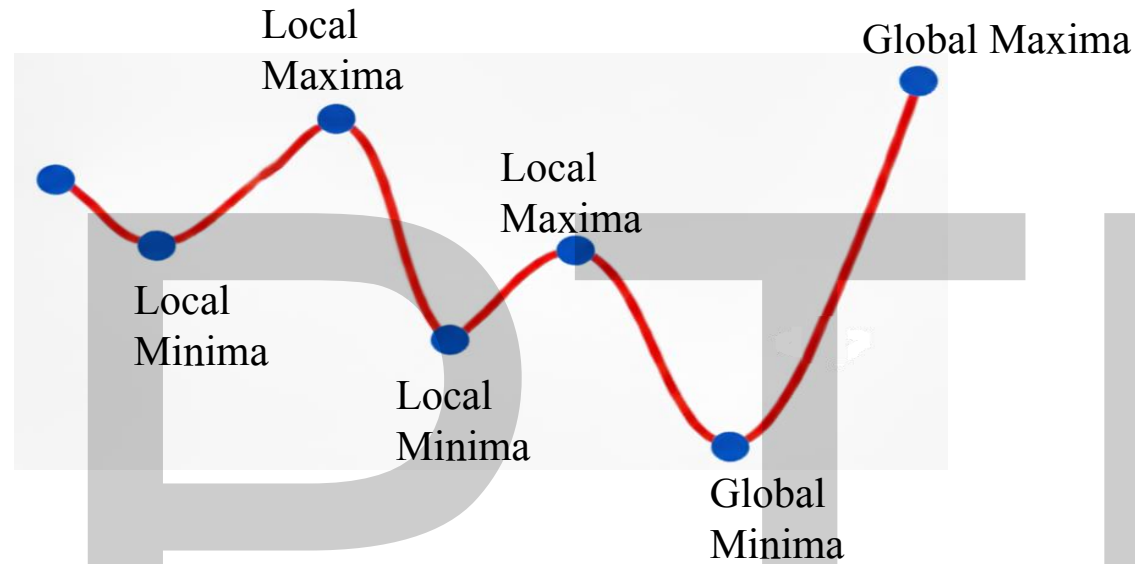
**Convex Function**

Gradient Descent converges to global minima in case of convex function



Multiple valley points are there

# Concept of Local and Global Minima and Maxima



## Global Minima

$$f(x^*) \leq f(x) \text{ for all } x \in \mathbb{R}$$

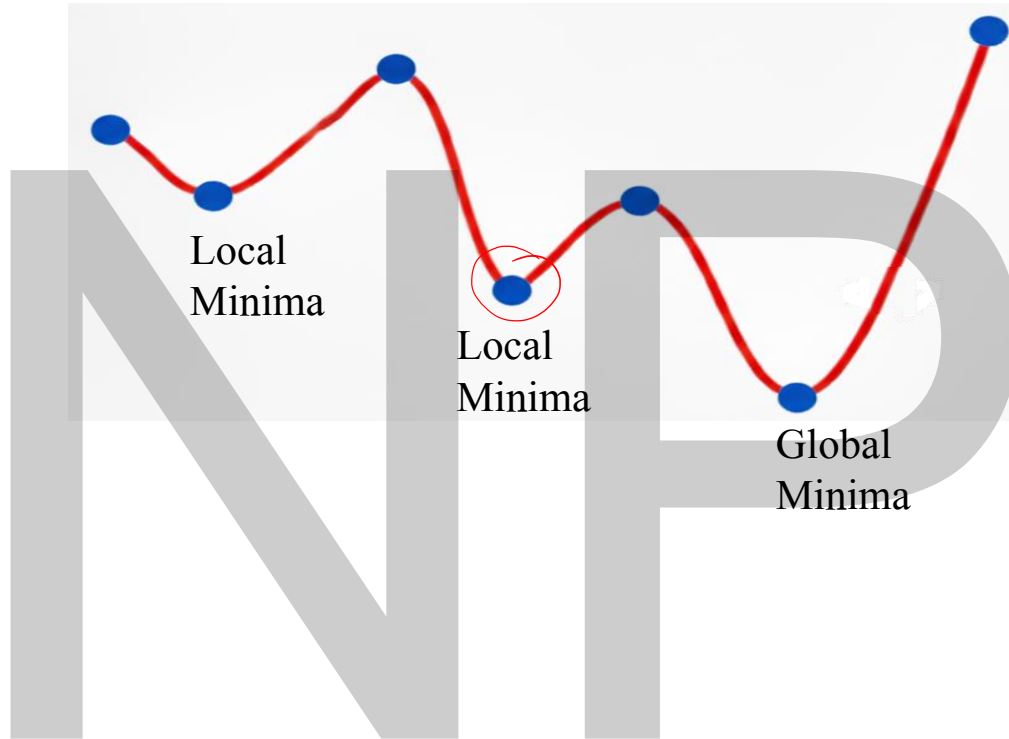
Minima over all possible values of  $x$

## Local Minima

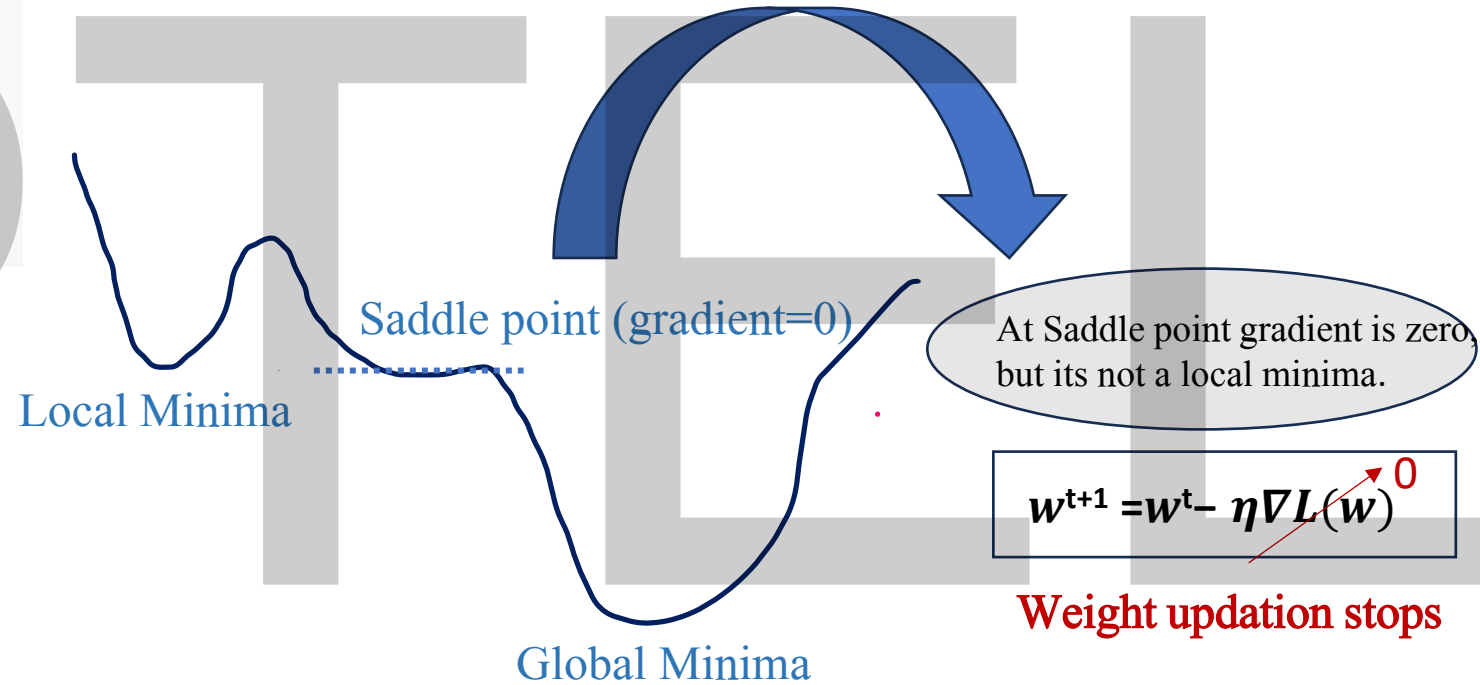
$$f(x^*) \leq f(x) \text{ for all } |x - x^*| < \delta \text{ if there exists a } \delta > 0$$

For all points close to  $x^*$ ,  $f(x) > f(x^*)$

# Local Minima, Global Minima and saddle point



Gradient Descent may settle in a local minima or a saddle point in case of a non convex function



At Saddle point gradient is zero, but its not a local minima.

$$w^{t+1} = w^t - \eta \nabla L(w) \quad \text{with } \nabla L(w) = 0$$

Weight update stops

How to solve this problem if  
getting stuck in local minima  
or at a saddle point ?



NPTEL

*Summary*

# NPTFEL

Next Session

Backpropagation Algorithm

NPTEL

Thank You